Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Data Engineering + Big Data + Automation + Data + Analytics +

Downsized, Not Defeated: How a Team of 4 Automated the Heavy Lifting

Anurag Sharma 10 min read · Aug 3, 2026



"We didn't have the luxury of panic. We had deadlines, broken pipelines, confused analysts, and an ML team waiting on clean data — all at once."

There is a particular kind of silence that follows a corporate downsizing announcement. Not peaceful. Not relieving. The kind where you sit at your desk, look at your task board, and slowly realize — the work did not leave with the people.

That is exactly where we found ourselves.

Our data engineering team went from a reasonably sized group to four people overnight. A 60% reduction on paper sounds clinical. In practice, it meant that everything we'd built systems, processes, and informal knowledge around — suddenly had to be operated by fewer hands, faster minds, and an uncomfortably tight margin for error.

This is the honest story of how we survived it. Not just survived — how we eventually built something better than what we had before.

The First Week: Reality Hits Hard

Before we talk about solutions, let us talk about what the first week actually looked like.

Our Slack was a war zone. The support team was pinging us about failed jobs before 9 AM. An analyst had opened three tickets about a report showing wrong numbers. Someone from the ML team sent a message that started with *"Hey, quick question…"* — which we learned is never quick. And somewhere in the backlog, buried under all of this noise, were two new pipeline development tickets that had been marked *"urgent"* three weeks ago.

We had four engineers. We had the workload of ten.

The first instinct — which we resisted — was to just start firefighting. Pick the loudest fire, put it out, move to the next. That approach, we knew from experience, is how teams burn out in three weeks and lose their best remaining people.

Instead, we did something counterintuitive: **we stopped working for a whole day and mapped the chaos.**

Mapping the Problem: Four Distinct Pain Points

After that full-day session, we had four clearly defined problem categories written on a whiteboard. Understanding them distinctly was the first real step toward solving them.

🔴 Problem 1 — The Support Tornado

Support teams reaching out constantly. This job failed. That job is still running. A file has not arrived. A downstream system is waiting on data. The messages came in waves — sometimes three about the same pipeline within an hour, from different people who had not spoken to each other.

Every interruption, however small, cost us roughly 20–30 minutes of context-switching. And we were getting 15–20 such interruptions a day across the team.

The math was brutal: four engineers, roughly five interruptions each per day, at 20 minutes each — that is shockingly seven hours of lost focused work. Every. Single. Day.

🟡 Problem 2 — The Analyst Question Loop

Analysts reaching out with data quality questions. Why is this report showing wrong numbers? This metric looks different from last week. Can you check if the pipeline ran correctly?

Most of these were not engineering problems. They were *visibility problems.* Analysts did not have the context to self-diagnose. They could not see pipeline status, data freshness timestamps, or upstream dependency states. So they did the only logical thing — they asked us.

The frustrating part? About 70% of these questions had answers that already existed somewhere in our systems. They just were not surfaced in a way analysts could access.

🟠 Problem 3 — The ML/DS Data Cleanliness War

ML and Data Science teams reaching out because data was not clean enough to train models, run experiments, or get reliable outputs.

This one stung a little differently. Because unlike support tickets, which were reactive, or analyst questions which were repetitive, data quality issues for ML had *downstream consequences.* A dirty dataset does not just block one report — it poisons a model, skews an experiment, delays a deployment.

And our ML colleagues, brilliant as they were, had a very different definition of "clean" than our pipelines were built to guarantee. There was a mismatch in expectations that nobody had ever formally resolved.

🔵 Problem 4 — Development Work Dying in the Backlog

New pipelines and new features kept getting deprioritized. Every day we spent firefighting was a day no new development happened. The backlog grew quietly — like a debt that accrues interest while you are too busy to look at your bank statement.

Leadership still expected delivery timelines to be met. Business units still needed new capabilities. But our capacity was entirely consumed by the first three problem categories. Development was not just slowing — it was effectively paused.

The Framework We Built: Divide, Automate, Conquer

We gave ourselves one rule before designing solutions: no solution would be accepted unless it either automated something or gave someone else the ability to self-serve. We could not hire our way out of this like we used to do previously. We had to engineer our way out.

Here's what we built, problem by problem.

Solving Problem 1: Killing the Support Tornado with Smart Alerting

The support interruptions were the highest-frequency problem, so we attacked it first.

Step 1 — Audit every interruption for two weeks

Before automating anything, we logged every support ping. Job name, time, reason, resolution time, whether it was truly an engineering issue or an operational one. Two weeks of data gave us a shocking picture: over 65% of support interruptions were about things that could have been caught and auto-resolved — or at minimum, auto-communicated — without engineer involvement.

Step 2 — Build a job health dashboard they could see themselves

We set up a lightweight observability layer using our existing orchestration tool, which is Apache Airflow combined with a simple Grafana dashboard.

Every pipeline got a public-facing status tile: green (running fine), yellow (delayed but within SLA), red (failed or breached SLA).

Support could now answer 80% of their own questions — *"Is this job running?"* — without pinging us.

Step 3 — Introduce AI-powered alert triage

This is where it got interesting. We used a small LLM-based Slack bot — think a GPT wrapper connected to our alerting system — that could:

- Receive a job failure alert
- Pull the last known error log
- Cross-reference a knowledge base we built from our top-30 recurring failure patterns
- Post a human-readable diagnosis directly into the support Slack channel, including a probable cause and a suggested first-action step

If the bot was 80%+ confident in the resolution, it would also auto-trigger a retry. If not, it escalated to the on-call engineer with a pre-written context summary.

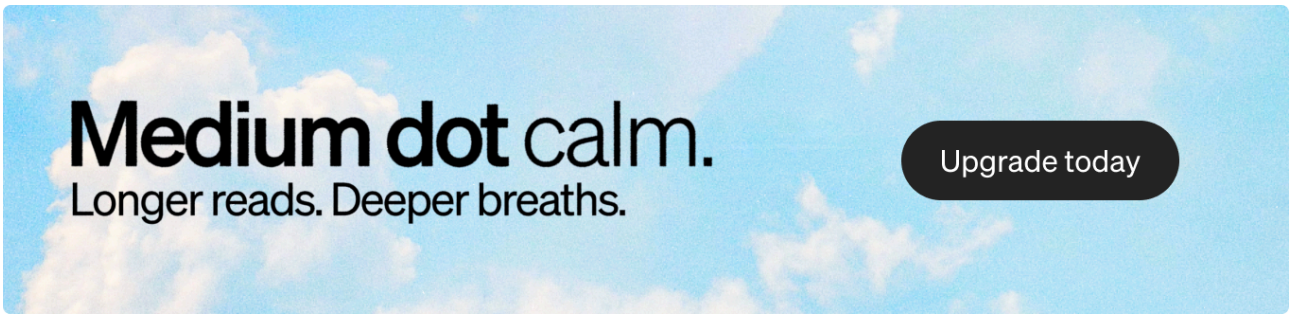
The result? Engineer interruptions from support dropped by 72% in the first month. Support teams were happier because they got faster answers. Engineers were happier because they could actually think.

Solving Problem 2: Giving Analysts Superpowers

The analyst question loop needed a different approach. These were not system failures — they were knowledge gaps. The solution was not more engineering; it was better information architecture.

Step 1 — Data freshness metadata on every table

We added three metadata columns to every table that analysts touched: `last_updated_at`, `pipeline_run_id`, and `data_quality_score`. These were populated automatically at the end of every pipeline run. Analysts could now instantly check if a table was stale before assuming the report was wrong.



Step 2 — A self-serve "Data Status" page

We built a single internal web page (literally a static HTML page refreshed every 15 minutes by a cron job) that showed every analyst-facing dataset: last run time, row count delta from prior run, quality check pass/fail status, and a link to the pipeline run log. No login required. Always current.

Step 3 — An AI-assisted data Q&A bot

This was our most ambitious move. We connected a RAG (Retrieval-Augmented Generation) based chatbot to our data documentation, pipeline run history, and schema definitions. Analysts could ask natural language questions:

> *"Why is the revenue report lower than last Tuesday?" "Did the customer table get updated today?" "What does the `net_revenue_adjusted` column actually measure?"*

The bot would retrieve relevant context from documentation and run history, then generate a clear explanation. It wasn't perfect — but it handled about 60% of analyst questions end-to-end, and for the remaining 40%, it handed engineers a pre-drafted context summary so they weren't starting from zero.

The cultural shift mattered too. We held one 30-minute session with the analyst team explaining how to use these tools. Framing it as *"we built this for you, not to brush you off"* made the adoption fast.

Solving Problem 3: Agreeing on "Clean" Before the Data Moves

The ML/DS data quality problem required the most fundamental mindset shift — because it was not just a technical problem. It was a contract problem.

Step 1 — Write a Data Quality Contract per dataset

We sat with the ML team for two hours and co-authored what we called a Data Quality Contract for every dataset they depended on. It specified:

- Acceptable null percentage per column
- Expected value ranges for key numerical fields
- Required uniqueness constraints
- Allowed categorical values for encoded fields
- Freshness SLA (e.g., "this table must be updated by 6 AM daily")

This document lived in our Git repo alongside the pipeline code. If the ML team's needs changed, they opened a pull request. It became a living agreement, not a verbal assumption.

Step 2 — Automated quality gates in every pipeline

We integrated Great Expectations (an open-source data quality framework) into every pipeline feeding ML datasets. Before a pipeline marked itself complete, it ran a suite of quality checks against the contract. If checks failed, the pipeline halted and alerted — it did not silently deliver bad data downstream.

This meant the ML team stopped receiving dirty data. And we stopped receiving *"why is my model behaving weirdly"* messages three days after the bad data had already done damage.

Step 3 — A data quality AI monitor

We added a lightweight anomaly detection layer — using a simple statistical model initially, then upgrading to an ML-based anomaly scorer — that ran against every ML-facing table daily. It learned normal patterns over 30 days, then flagged deviations automatically. If a column that normally has 2% nulls suddenly showed 18% nulls, the system flagged it, generated a probable cause (upstream pipeline changed, source schema altered, etc.), and sent a proactive alert — before anyone asked.

No more surprise dirty data. No more reactive panic.

Solving Problem 4: Protecting Development Time Like It Was Sacred

This was the hardest problem — not technically, but organizationally. You can automate an alert. You cannot automate discipline and prioritization without cultural buy-in.

Step 1 — The "Time Budget" Framework

We explicitly allocated team hours like a budget, every single week. Our rule:

Category	Weekly Hours Allocated
Support & incidents	20%
Analyst support	15%
ML/DS support	15%
Development work	**50%**

The first two weeks, we tracked actual vs. allocated. The results were embarrassing — development was getting less than 10% of our time. But seeing the number made it real. We started protecting development blocks like meetings: they were on calendars, they had a do-not-disturb status, and they were non-negotiable unless a true production incident occurred.

Step 2 — Two-person rotation for firefighting

At any given time, two engineers were "on support rotation" and two were "in development mode." The support pair handled all incoming queries, tickets, and escalations. The development pair had full heads-down time. We switched every week.

This single change — more than any automation — was what finally allowed development work to resume at a meaningful pace.

Step 3 — AI-assisted code and pipeline scaffolding

For the actual development work, we leaned into AI coding tools aggressively. GitHub Copilot for boilerplate pipeline code. A custom prompt library for generating transformation logic, schema migration scripts, and

unit test skeletons. Tasks that previously took a full day now took half a morning.

We also built a pipeline template library — reusable, parameterized templates for the five most common pipeline patterns we built. New pipelines went from blank page to deployable skeleton in under an hour. Our junior engineers especially benefited — less time on scaffolding meant more time on actual logic and edge-case handling.

What the Numbers Looked Like Three Months Later

We ran a retrospective ninety days after implementing all of this. Here's what the data showed:

Metric	Before	After
Daily engineer interruptions	18–22	4–6
Analyst tickets per week	30+	8–10
ML data quality incidents per month	12	2
New pipelines deployed per month	1–2	6–8
Engineer-reported burnout score (1–10)	8.2	4.7

The last row matters most to us. The work did not shrink. But the *chaos* did. And it turns out, people can handle hard work — what they cannot handle is hard work with no signal, no structure, and no sense of progress.

The Unexpected Benefits Nobody Planned For

A few things happened that we did not anticipate.

The documentation got better. Because we had to build self-serve tools, we had to document our systems properly for the first time. The AI bots needed

a knowledge base. The data contracts needed to be written down. Six months in, our documentation was the best it had ever been — not because someone mandated it, but because automation demanded it.

Mid-Senior Engineers grew faster. With templates, AI coding assistance, and cleaner processes, our junior team members could own complete pipeline builds without needing constant hand-holding. The constraints created clarity about responsibilities that a larger team had never formalized.

We were ready to scale back up. When the company eventually began hiring again, we weren't inheriting a mess. We were offering a well-documented, automated, high-visibility system that new engineers could onboard to in days rather than months.

For Anyone Standing Where We Stood

If you are a small data engineering team staring at a workload that does not match your headcount, here is the condensed version:

- Categorize your chaos before you start solving it. Firefighting all four problem types with the same approach does not work.
- Automation buys you time. Visibility buys you trust. Support teams and analysts stop interrupting you when they can see what's happening themselves.
- Agreements prevent incidents. The Data Quality Contract idea sounds boring — it is one of the most impactful things we ever did.
- Protect development time explicitly, or it simply will not happen. Good intentions are not a calendar block.
- AI is a force multiplier, not a magic wand. Every AI tool we built worked because we gave it clean inputs, clear scope, and a feedback loop. None of them worked out of the box.

Downsizing is hard. Being the ones who stay is hard. But small, coordinated teams that build well can move faster and build better than large, unstructured ones.

We did not just survive the cut. We built the best version of our team under that pressure.

Data Engineering + Big Data + Automation + Data + Analytics +



Written by Anurag Sharma

149 followers · 8 following

Data Engineering Specialist with 11+ Exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!

Follow